알고리즘 적용 기획서 #1

■ 내용 : SHA-256 해시 함수를 이용한 사용자 비밀번호 보안 저장 서비스

■ 적용 알고리즘 : SHA-256 (Secure Hash Algorithm 256-bit)

■ 알고리즘 개요

SHA-256 알고리즘은 임의의 길이를 가진 데이터를 고정된 256 비트(32 바이트) 길이의 데이터로 변환하는 단방향 암호화 해시 함수입니다. 입력값이 아주 조금만 달라져도 결과값이 완전히 바뀌기 때문에 결과값에서 입력값을 역추적하는 것이 거의 불가능합니다. 이를 통해 사용자 비밀번호를 평문으로 저장하지 않고 해시화된 문자열로 저장하여 DB 유출 시에도 보안성을 유지할 수 있습니다.

■ 적용 서비스 : 사용자 회원가입 및 로그인 보안 인증 모듈

■ 코드 경로 : [project root]/js/sha256.js

■ 적용 서비스 개발 개요

저장소에 단방향 암호화 된 값만 저장되어 있다면 비밀번호가 유출되더라도 평문을 공격자에게 노출하지 않을 수 있어 보안성을 높일 수 있습니다.

아래 코드는 NIST 의 공식 명세서(https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.180-4.pdf)에서 지시하는 대로 작성되었습니다.

1. **초기값 설정(Initialize Hash Values)** : SHA-256 은 8 개의 32 비트 워드를 사용하여 해시값을 초기화 합니다.
   아래 코드에서 **hash** 는 첫 8 개의 소수(2, 3, 5, 7, 11, 13, 17, 19)의 제곱근 중 소수점 아래 32 비트 값입니다.
   k 는 첫 64 개의 소수의 세제곱근의 소수점 아래 32 비트 값입니다.(공식 명세서의 4.2.2 섹션에 k 값들의 목록이 있습니다.)
   isPrime 은 특정 숫자가 소수인지 아닌지를 체크하는 불리언 맵처럼 작동합니다. 어떤 숫자가 소수로 판정되면, 해당 숫자의 배수를 전부 **isPrime** 에 소수가 아니라고 등록합니다.
   i 를 313 까지 for 문으로 돌리는 이유는 64 번째 소수가 311 이기 때문입니다.
   소수점 아래 32 비트 값을 빠르게 구하기 위해 2 의 32 승(변수 maxWord)를 곱합니다. 이후 **| 0 연산**을 하면 숫자가 32 비트 정수로 강제

변환되고, 이 때 2 의 32 승을 넘어가는 부분은 오버플로우가 되어 사라지고, 소수점 아래였던 부분들만 32 비트 정수로 남게 됩니다.

```
var mathPow = Math.pow;
var maxWord = mathPow(2, 32);

//* 초기 해시 값 (첫 8 개 소수의 제곱근의 소수 부분)
var hash = [], k = [];
var primeCounter = 0;
var isPrime = {};
for (var candidate = 2; primeCounter < 64; candidate++) {
  if (!isPrime[candidate]) {
    for (i = 0; i < 313; i += candidate) isPrime[i] = candidate;
    if (primeCounter < 8)
      hash[primeCounter] = (mathPow(candidate, 0.5) * maxWord) | 0;
    k[primeCounter] = (mathPow(candidate, 1 / 3) * maxWord) | 0;
    primeCounter++;
  }
}
```

2. **데이터 규격화 (Padding)**: SHA-256 은 데이터를 512 비트(64 바이트) 단위로 잘라서 처리합니다.
   입력 데이터가 딱 **64 바이트의 배수가 아닐 경우를 고려하여 부족한 부분을 채워줍니다**. 채워주는 수는 `₩x80`으로, 이진수로 10000000 입니다. (명세서의 5.1.1 섹션에 명시된 숫자입니다.)
   이후 전체 길이를 64 로 나눈 나지가 65 바이트(448 비트)가 될 때까지 0 을 붙입니다. 56 바이트인 이유는 마지막에 메시지의 원래 길이를 기록할 8 바이트 공간을 남기기 위함입니다. 길이를 적음으로서 충돌 회피와 패딩과 진짜 데이터의 구분이 가능해집니다.

```
ascii += "\x80"; // 패딩: 1 비트 추가
while ((ascii[lengthProperty] % 64) - 56) ascii += "\x00"; // 패딩: 0 으로 채움
```

자바스크립트 문자열은 1 바이트 단위이지만
알고리즘 연산은 4 바이트 단위로 돌아가기 때문에 4 바이트씩 하나의 숫자로 묶습니다.

```
for (i = 0; i < ascii[lengthProperty]; i++) {
  j = ascii.charCodeAt(i);
  if (j >> 8) return; // ASCII 범위를 벗어나는 경우 처리
  words[i >> 2] |= j << ((3 - (i % 4)) * 8);
}
```

마지막 64 비트에 메시지 길이를 기록합니다. (SHA-256 표준)

```
words[words[lengthProperty]] = (asciiBitLength / maxWord) | 0;
words[words[lengthProperty]] = asciiBitLength | 0;
```

3. **데이터 믹싱 연산 (64 Rounds)**: 정해진 64 번의 단계 동안 비트 단위로 숫자를 밀고(Shift), 섞고(Rotate), 논리 연산(AND, OR, XOR)을 수행했습니다. 이 과정에서 아주 미세한 비밀번호 차이도 완전히 다른 결과값으로 변하게 됩니다.

```
for (j = 0; j < words[lengthProperty]; ) {
  var w = words.slice(j, (j += 16));
  var oldHash = hash;
  hash = hash.slice(0, 8);

  for (i = 0; i < 64; i++) {
      var w15 = w[i - 15],
      w2 = w[i - 2];

    var a = hash[0],
      e = hash[4];
    var temp1 =
      hash[7] +
      (rightRotate(e, 6) ^ rightRotate(e, 11) ^ rightRotate(e, 25)) + // Σ1
      ((e & hash[5]) ^ (~e & hash[6])) + // Ch
      k[i] +
      (w[i] =
        i < 16
          ? w[i]
          : (w[i - 16] +
              (rightRotate(w15, 7) ^ rightRotate(w15, 18) ^ (w15 >>> 3)) +
// σ0
              w[i - 7] +
              (rightRotate(w2, 17) ^ rightRotate(w2, 19) ^ (w2 >>> 10))) |
// σ1
            0);

    var temp2 =
      (rightRotate(a, 2) ^ rightRotate(a, 13) ^ rightRotate(a, 22)) + // Σ0
      ((a & hash[1]) ^ (a & hash[2]) ^ (hash[1] & hash[2])); // Maj

    hash = [(temp1 + temp2) | 0].concat(hash);
    hash[4] = (hash[4] + temp1) | 0;
  }
```

```
  for (i = 0; i < 8; i++) hash[i] = (hash[i] + oldHash[i]) | 0;
}
```

먼저 512 비트 블록을 가져옵니다.
그리고 이전에 작업해둔 hash 를 oldHash 변수에 백업합니다. oldHash 는 현재 블록 연산이 끝난 뒤(현재 블록에 64 회 라운드 돌아간 뒤) 마지막에 더해집니다. 이번에 작업할 8 개 변수를 준비합니다.

```
var w = words.slice(j, (j += 16));
var oldHash = hash;
 hash = hash.slice(0, 8);
```

SHA-256 은 **한 블록당 64 번의 라운드**를 돌립니다.
명세서 6.2.2 섹션에 각 라운드의 로직이 정리되어 있습니다.

w15 에는 w[i-15]의 값이 담기는데, js 에서는 불가능한 인덱스에 접근시 undefined 가 반환되며 오류가 나지 않습니다. 이후, **16 번째 이후에 w15 가 사용**되며, **w15** 변수에는 이전 단계의 메시지 조각이 담겨있습니다. **8 개의 변수**(위에서 작업을 위해 8 개로 잘라 준비한 **hqsh 변수**) 중 **첫 번째는 a** 에, **두 번째는 e** 에 저장되어 이후 사용됩니다.. (SHA-256 표준)

```
for (i = 0; i < 64; i++) {
  var w15 = w[i - 15], w2 = w[i - 2];
  var a = hash[0],  e = hash[4];
```

메시지 확장 및 Temp1 계산을 진행합니다.
hash[7]은 현재의 h(SHA-256 계산에서 내부적으로 사용되는 8 개의 32 비트 변수를 공식 명세서에서 편의상 a, b, c, d, e, f, g, h 로 이름 붙임)를 나타납니다. SHA-256 의 한 라운드가 돌아갈 때 변수들은 컨베이어 벨트처럼 새로운 계산 결과가 a 위치로 들어가면, 기존의 a 는 b 가 되고 , b 는 c 가 되는 식으로 움직이는데, 가장 마지막 칸이 h 입니다. h 는 이전 라운드가 모두 진행되고 가장 마지막 값을 뜻하며, 이 값을 temp1 에 더함으로 서 이전 라운드의 결과물이 현재 라운드의 새로운 a 와 e 를 만드는데 기여합니다.
rightRotate 는 e 를 비트 회전시켜 섞는 함수로 SHA-256 표준에 정의된 Σ1 함수입니다.
((e & hash[5]) ^ (~e & hash[6]))에서 e 비트가 1 이면 f 를, 0 이면 g 를 선택합니다.
w[i]에 대입되는 삼항연산자에서, i 가 16 보다 작을 경우 입력받은

메시지를 그대로 사용하고, 16 보다 크거나 같을 경우 기존 메시지들을 비트 회전 시키고 더해 새로운 32 비트 단어를 생성하여 저장합니다.

```
var temp1 =
 hash[7] +
 (rightRotate(e, 6) ^ rightRotate(e, 11) ^ rightRotate(e, 25)) + // Σ1
 ((e & hash[5]) ^ (~e & hash[6])) + // Ch
 k[i] +
  (w[i] = < 16 ?
  w[i] :
  (w[i - 16] + rightRotate(w15, 7) ^ rightRotate(w15, 18) ^ (w15 >>> 3)) +
// σ0
  w[i - 7] +
  (rightRotate(w2, 17) ^ rightRotate(w2, 19) ^ (w2 >>> 10))) | // σ1
  0);

(…)

function rightRotate(value, amount) {
    return (value >>> amount) | (value << (32 - amount));
  }
```

Temp2 을 연산합니다. A 를 비트 회전시켜 섞습니다.
a, b, c 세 값 중 비트 단위로 더 많이 나타나는 값(0 또는 1)을 고릅니다. 이는 데이터 확산을 돕습니다. 데이터 확산은 작은 변화가 크게 바뀌는 것으로, 여기서 선택된 값이 다음 변수인 b, c… 에 영향을 미치고, 다음 라운드를 넘어 전체 해시 결과에도 영향을 미칩니다. 이를 통해 입력값과 출력값 사이 통계적 패턴이 지워지고, 공격자가 해시를 통해 평문의 예측하는 것이 더욱 어려워집니다.

```
var temp2 =
  (rightRotate(a, 2) ^ rightRotate(a, 13) ^ rightRotate(a, 22)) + // Σ0
  ((a & hash[1]) ^ (a & hash[2]) ^ (hash[1] & hash[2])); // Maj(a,b,c)
```

변수를 업데이트 합니다.
앞에서 구한 temp1 과 temp2 를 더하여 새로운 a 값으로 저장하고, 기존의 a 는 b 로, 기존의 b 는 c 로 … h 까지 진행합니다. (코드 상으로는 새로운 a 로 시작하는 원소 1 개짜리 배열 뒤에 기존 hash 배열을 이어붙이는 모양)
hash[4] (e 값)에 temp1 을 더합니다. 보안성을 향상시키기 위한 트릭입니다.

```
hash = [(temp1 + temp2) | 0].concat(hash);
hash[4] = (hash[4] + temp1) | 0;
```

64 번의 라운드가 끝난 뒤,
이전 블록 결과(이 블록의 시작 전 상태)와 결과물을 더합니다. 이 값은 다음 512 비트 블록의 초기값이 되며, 마지막 512 비트 블록이었다면 결과 hash 가 됩니다.

```
for (i = 0; i < 8; i++) hash[i] = (hash[i] + oldHash[i]) | 0;
```

4. **고정된 결과 도출**: 모든 계산이 끝난 뒤, 8 개의 32 비트 숫자 형태로 저장된 해시 결과값을 64 글자의 16 진수 문자열로 변환합니다. 함수 초기에 1 바이트 문자열 4 개를 합쳐 4 바이트 숫자로 바꾼 것의 역입니다.

```
for (i = 0; i < 8; i++) {
  for (j = 3; j + 1; j--) {
    var b = (hash[i] >> (j * 8)) & 255;
    result += (b < 16 ? "0" : "") + b.toString(16);
  }
}
```

5. **로그인 검증 프로세스**: 서버 DB 에는 비밀번호 원본이 아닌 '암호화된 결과값'만 저장합니다. 로그인을 할 때마다 사용자가 입력한 값을 실시간으로 암호화하여 DB 값과 일치하는지 대조하는 안전한 인증 시스템을 구현했습니다.

```
const passHash = sha256_pure(password);
      if (!user || user.password !== passHash) {
        return {
          ok: false,
          message: "아이디 또는 비밀번호가 올바르지 않습니다.",
        };
      }
```

■ 알고리즘이 적용된 모습

Key	Value
ssafy_favorites_test	[{"sido":"서울특별시","sigungu":"종로구","emd":"","lawdCd":"11110","key":"11110|서...
ssafy_session	{"userId":"test","loginAt":"2026-04-17T05:56:49.657Z"}
ssafy_users	[{"userId":"test","password":"9f86d081884c7d659a2feaa0c55ad015a3bf4f1b2b0b822...

알고리즘 적용 기획서 #2

■ 내용 : 병합 정렬(Merge Sort)을 이용한 다중 필터 기반 부동산 매물 정렬 서비스

■ 적용 알고리즘 : 병합 정렬 (Merge Sort)

■ 알고리즘 개요

병합 정렬은 분할 정복(Divide and Conquer) 방식을 사용하는 비교 기반 정렬 알고리즘입니다. 배열의 길이가 1 이 될 때까지 반으로 나눈 후, 정해진 우선순위에 따라 다시 합치면서 정렬을 완성합니다. 최악, 최선, 평균의 경우 모두 O(n log n)의 일관된 시간 복잡도를 보장하며, 동일한 값을 가진 요소의 순서가 유지되는 '안정 정렬(Stable Sort)'이라는 특징이 있어 다중 조건 정렬에 적합합니다.

■ 적용 서비스 : SSAFY HOME 매물 검색 결과 정렬 시스템

■ 코드 경로 : [project root]/js/merge-sort.js

■ 적용 서비스 개발 개요

수천 개의 부동산 매물을 사용자가 원하는 기준(가격, 면적 등)에 맞춰 즉시 재배치하기 위해 다음과 같은 과정을 거쳐 구현하였습니다.

1. **분할 및 정복 (Split & Conquer)**: 수많은 매물 리스트를 데이터가 1 개가 남을 때까지 절반씩 계속해서 쪼갰습니다. 이는 큰 문제를 작은 단위로 나누어 해결함으로써 대량의 데이터를 처리할 때 발생하는 과부하를 방지하기 위함입니다.

```
const mid = Math.floor(arr.length / 2);
const left = mergeSort(arr.slice(0, mid), compFunction);
const right = mergeSort(arr.slice(mid), compFunction);
```

2. **유연한 비교 기준 설정 (Comparator)**: 가격이 높은 순, 면적이 넓은 순 등 사용자가 선택한 기준에 따라 정렬 기준을 자유롭게 바꿀 수 있도록 '비교 전용 함수'를 설계했습니다. 이를 정렬 알고리즘에 끼워 넣는 방식으로 개발하여 코드의 재사용성을 높였습니다.

```
function mergeSort(arr, compFunction)
```

3. **데이터 병합 (Merge)**: 쪼개진 매물들을 다시 하나로 합치면서, 설정한 기준에 맞춰 하나씩 순서를 비교하여 배치했습니다. 이때 값이 같은 매물은 원래 있던 순서를 그대로 유지하게 하여 데이터의 신뢰도를 높였습니다.

```
function merge(left, right, compFunction) {
  let result = [];
  let i = 0;
  let j = 0;

  // 양쪽 배열을 비교하며 결과 배열에 순서대로 삽입
  while (i < left.length && j < right.length) {
    // compare 함수의 결과가 0 이하이면 왼쪽 요소가 먼저 옴
    if (compFunction(left[i], right[j]) <= 0) {
      result.push(left[i]);
      i++;
    } else {
      result.push(right[j]);
      j++;
    }
  }

  // 남은 요소들을 결과 배열에 붙임
  return result.concat(left.slice(i)).concat(right.slice(j));
}
```